

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



Curso:

Programación orientada a objetos

Proyecto:

Granja AgroSan

Autores:

- **Montesinos Parisaca, Tupac Russell** **20244839D**
- **Romero Luna. Kevin Stiven Jefferson** **20231112C**
- **Serrano Arone, Adrian Ruben** **20221190A**

Lima, 2025

Dedicatoria

A nuestros queridos padres, con amor y gratitud dedicamos este trabajo. Ustedes han sido faros de fortaleza, ejemplo y guía constante. Gracias por sus sacrificios, por su confianza en nosotros y por motivarnos en cada paso de nuestro camino académico.

Índice General

- Resumen
- Abstract
- Introducción
- 1. Objetivos
 - 1.1. Objetivo General
 - 1.2. Objetivos Específicos
- 2. Requerimientos
 - 2.1. Requerimientos Funcionales
 - 2.2. Requerimientos no funcionales
 - 2.3. Alcances
 - 2.4. Límites
- 3. Análisis de requerimientos
 - 3.1. Identificación de actores
 - 3.2. Casos de uso
- Modelo Conceptual
- Modelo Relacional
- Diccionario de Datos
- IMPLEMENTACIÓN DE LA BASE DE DATOS
 - Creación de la Base de Datos
- CÓDIGO FUENTE
- CONCLUSIONES
- LECCIONES APRENDIDAS
- BIBLIOGRAFÍA
- ANEXOS
 - Anexo 1: Diagrama de Arquitectura del Sistema
 - Anexo 2: Colección de Postman
 - Anexo 3: Scripts de Base de Datos

Resumen

El proyecto se centra en la creación de un sistema integral de gestión agrícola llamado "**AgroSan**" que será usado para el desarrollo de una aplicación web destinada a optimizar la operación de una granja dedicada al cultivo de hortalizas y frutas. El sistema utiliza tecnologías modernas como Spring Boot para la lógica del negocio, SQL Server para la persistencia de datos y Postman para las pruebas de integración.

El modelo de base de datos fue diseñado con un enfoque relacional, asegurando integridad entre los datos y soporte para futuras ampliaciones del proyecto, esto podrá ser observado con mayor detalle en el diccionario de datos del proyecto. Por otro lado, a lo largo del desarrollo, se hizo hincapié en las buenas prácticas de programación, como la implementación de transacciones seguras en la base de datos y la validación rigurosa de datos para asegurar la integridad de la información.

Asimismo, el sistema realizado para este proyecto permite gestionar de manera correcta el ciclo completo de producción agrícola: desde el registro de siembras, programación de actividades de mantenimiento, registro de cosechas hasta el control de ventas. Incluye además un sistema de alertas que notifica automáticamente sobre fechas importantes como cosechas próximas.

Por último, se darán algunas conclusiones y lecciones aprendidas que se adquirió a lo largo del desarrollo del proyecto.

Palabras clave: Sistema agrícola, gestión de cultivos, Spring Boot, SQL Server, siembra, cosecha, alertas.

Abstract

The project focuses on the creation of a comprehensive agricultural management system called **"AgroSan"** that will be used for the development of a web application aimed at optimizing the operation of a farm dedicated to growing vegetables and fruits. The system uses modern technologies such as Spring Boot for business logic, SQL Server for data persistence, and Postman for integration testing.

The database model was designed with a relational approach, ensuring data integrity and support for future project expansions. This can be seen in greater detail in the project's data dictionary. Throughout development, emphasis was placed on good programming practices, such as the implementation of secure database transactions and rigorous data validation to ensure information integrity.

Furthermore, the system developed for this project allows for the proper management of the complete agricultural production cycle: from seeding records, maintenance activity scheduling, harvest records to sales control. It also includes an alert system that automatically notifies about important dates such as upcoming harvests.

Finally, some conclusions and lessons learned acquired throughout the development of the project will be given.

Keywords: Agricultural system, crop management, Spring Boot, SQL Server, seeding, harvest, alerts.

Introducción

El sector agrícola desempeña un rol crucial en el desarrollo económico y alimentario de cualquier nación, siendo responsable de la producción de alimentos básicos para la población. Sin embargo, en un entorno agrícola moderno y competitivo, las granjas enfrentan desafíos significativos en cuanto a la gestión eficiente de sus operaciones. Estos desafíos incluyen el control preciso de siembras, la programación adecuada de actividades de mantenimiento, el seguimiento de cosechas y la gestión de ventas, todo ello con el fin de optimizar la productividad y reducir pérdidas.

En este contexto, surge la necesidad de una solución tecnológica que permita centralizar y optimizar la gestión de cultivos, actividades agrícolas y producción dentro de una granja. Este proyecto se enfoca en el diseño e implementación de una aplicación web llamada "**AgroSan**", utilizando el framework Spring Boot, que sirva como sistema de gestión integral para granjas dedicadas al cultivo de hortalizas y frutas. Esta aplicación se conectará con una base de datos relacional bien estructurada, que permita almacenar, consultar y actualizar de manera eficiente los registros de siembras, actividades, cosechas y ventas realizadas.

El sistema busca resolver problemas comunes como la falta de trazabilidad en las operaciones agrícolas, la dificultad para programar actividades de mantenimiento, la pérdida de información sobre rendimientos de cosechas y la falta de alertas automáticas para fechas críticas. Además, facilitará la planificación estratégica de cultivos, el control de inventarios de semillas y cosechas, y la generación de reportes para toma de decisiones.

El objetivo general del proyecto es desarrollar una aplicación web para centralizar la gestión de cultivos agrícolas en una granja, mediante la implementación de servicios eficientes que respondan a los requerimientos funcionales definidos. Esto incluye el registro de siembras, la programación de actividades, el control de cosechas y ventas, con un sistema de alertas que mejore la eficiencia operativa.

En resumen, este proyecto demuestra cómo el uso de tecnologías modernas como Spring Boot y bases de datos relacionales puede transformar y profesionalizar la administración agrícola, promoviendo un entorno más eficiente, productivo y sostenible. La implementación de esta solución permitirá a las granjas optimizar sus operaciones, reducir pérdidas y mejorar su competitividad mediante decisiones basadas en datos precisos y actualizados.

1. Objetivos

1.1. Objetivo General

Desarrollar un sistema web de gestión agrícola integral llamado **"AgroSan"** que centralice y optimice el registro, seguimiento y control del ciclo completo de producción agrícola (siembra, mantenimiento, cosecha y venta) en una granja, mediante una plataforma eficiente, segura y escalable que permita mejorar la trazabilidad de las operaciones y la toma de decisiones estratégicas.

1.2. Objetivos Específicos

- Diseñar e implementar un módulo para el registro y administración de siembras, que permita almacenar información detallada sobre tipo de cultivo, parcela asignada, cantidad sembrada, empleado responsable y fechas relevantes (siembra y cosecha estimada).
- Desarrollar una funcionalidad de programación y seguimiento de actividades agrícolas, que permita calendarizar tareas como riego, fertilización, aplicación de pesticidas, asociándolas a cultivos específicos con control de cumplimiento y recordatorios automáticos.
- Implementar un sistema de registro de cosechas y control de producción, que permita documentar los rendimientos obtenidos, actualizar automáticamente los inventarios de cosecha y generar estadísticas de productividad por cultivo y parcela.
- Gestionar el control de ventas y distribución de productos, permitiendo registrar transacciones comerciales con compradores, cantidades vendidas, precios y fechas, manteniendo un seguimiento preciso del flujo de productos desde la cosecha hasta la venta.
- Desarrollar un sistema de alertas y notificaciones automáticas que avise sobre fechas críticas como cosechas próximas, actividades pendientes o niveles bajos de stock de semillas e insumos.
- Garantizar la usabilidad, rendimiento, seguridad y confiabilidad del sistema, mediante una interfaz intuitiva, tiempos de respuesta adecuados, mecanismos de autenticación, respaldo automático de datos y control de integridad referencial.
- Diseñar una arquitectura escalable y preparada para integraciones futuras, como sensores IoT, módulos de análisis predictivo o integración con sistemas de gestión financiera, asegurando que el sistema pueda adaptarse al crecimiento de la granja.

2. Requerimientos

2.1. Requerimientos Funcionales

- **RF1: Registrar siembras con información básica.**
 - *Criterio de aceptación: Al ingresar datos válidos de una nueva siembra, el sistema guarda el registro completo y lo incluye en la lista de siembras activas, validando existencia de parcela y empleado, y calculando automáticamente la fecha estimada de cosecha según el tipo de cultivo.*
- **RF2: Programar y gestionar actividades agrícolas.**
 - *Criterio de aceptación: Para un cultivo registrado, el usuario puede programar actividades específicas indicando fecha, tipo de actividad y cultivo/parcela asociados; el sistema registra la programación y permite marcar actividades como completadas, mostrando recordatorios de actividades pendientes.*
- **RF3: Generar alertas y recordatorios automáticos.**
 - *Criterio de aceptación: El sistema monitorea constantemente fechas críticas y genera notificaciones automáticas, por ejemplo, alertas 7 días antes de fechas estimadas de cosecha indicando cultivo, parcela y acciones preparatorias necesarias para la recolección.*
- **RF4: Registrar cosechas y rendimientos obtenidos.**
 - *Criterio de aceptación: Al completar una cosecha, el usuario registra la cantidad obtenida, fecha de cosecha y empleado responsable; el sistema valida que exista una siembra activa en la parcela, actualiza automáticamente el stock de cosecha y marca el cultivo como cosechado.*
- **RF5: Gestionar ventas y distribución de productos.**
 - *Criterio de aceptación: Tras registrar una cosecha, el sistema permite registrar ventas indicando comprador, cantidad vendida, precio unitario y fecha; valida existencia de stock suficiente, calcula totales automáticamente y actualiza inventarios, permitiendo seguimiento del flujo de productos desde cosecha hasta venta final.*

2.2. Requerimientos no funcionales

- **Usabilidad:** La aplicación debe ser intuitiva para usuarios agrícolas con conocimientos básicos de informática. Ofrece interfaces simples con iconos claros, formularios sencillos y navegación lógica.
- **Rendimiento:** Las consultas de siembras activas, actividades pendientes y reportes deben responder en menos de 2 segundos.
- **Confiabilidad:** Los datos agrícolas críticos deben mantenerse integros mediante transacciones seguras y respaldos automáticos. La integridad referencial se mantiene estrictamente.
- **Seguridad:** Se implementa autenticación de empleados con credenciales únicas. Diferentes niveles de acceso (empleados básicos vs administradores).
- **Escalabilidad:** El sistema está preparado para crecer, soportando múltiples parcelas, tipos de cultivo y empleados. La arquitectura permite futuras integraciones (IoT, análisis predictivo).

2.3. Alcances

- Funcionalidades para registro completo del ciclo agrícola: siembra, actividades de mantenimiento, cosecha y ventas.
- Sistema de alertas automáticas para fechas críticas.
- Control de inventarios de semillas y cosechas.
- Gestión de empleados y parcelas.
- Reportes y consultas de producción.
- Interfaz REST API completa para integración futura.

2.4. Límites

- Desarrollo de aplicaciones móviles nativas.
- Integración con sensores IoT o hardware agrícola.
- Módulos de análisis predictivo avanzado.
- Sistema de gestión financiera integrado.
- Soporte multiidioma.
- Portal web público para compradores.
- El proyecto debe completarse dentro del periodo académico establecido.
- Desarrollo académico sin fines comerciales.
- Funciona en entorno local o red cerrada de la granja.
- No incluye integración con servicios externos de clima o mercados.
- Implementación limitada a hortalizas y frutas (no incluye cultivos extensivos).
- No contempla funcionalidades de gestión financiera detallada.

3. Análisis de requerimientos

3.1. Identificación de actores

CÓDIGO	NOMBRE	DESCRIPCIÓN
ACT01	Empleados	Personas que registran actividades agrícolas y gestionan el sistema

3.2. Casos de uso

CÓDIGO	ACTOR	NOMBRE	DESCRIPCIÓN
CU01	ACT01	Registro de siembras	Permite registrar nuevas siembras con toda su información
CU02	ACT01	Programación de actividades agrícolas	Permite programar actividades de mantenimiento para cultivos
CU03	ACT01	Registro de cosechas	Permite registrar cosechas obtenidas
CU04	ACT01	Registro de ventas	Permite registrar ventas de productos cosechados
CU05	ACT01	Generación de alertas	Sistema genera alertas automáticas para fechas críticas

3.2.1. CU01 - Registro de siembras

Especificación del caso de uso

Código	CU01

Nombre	Registro de siembras
Descripción	Permite registrar una nueva siembra en el sistema con sus datos esenciales como tipo de cultivo, parcela, empleado, cantidad sembrada y fecha automática.
Actores	ACT01
Requerimiento asociado	RF1: Registrar siembras con información básica
Precondición	El empleado debe estar autenticado en el sistema.

Secuencia normal

Paso	Acción
1	El empleado accede al formulario y completa los datos de la nueva siembra.
2	El sistema valida que la parcela exista y esté activa.
3	El sistema valida que el empleado exista y esté activo.
4	El sistema valida que el tipo de cultivo exista.
5	El sistema valida que haya stock suficiente de semillas.
6	El sistema registra la siembra en la base de datos con fecha automática.
7	El sistema calcula y registra fecha estimada de cosecha.
8	El sistema actualiza el stock de semillas.
9	El sistema confirma el registro exitoso al empleado.

--	--

Postcondición	La siembra queda registrada y el stock actualizado.
----------------------	---

Excepciones

Paso	Acción
1	Si la parcela no existe o no está activa, lanza excepción.
2	Si el empleado no existe o no está activo, lanza excepción.
3	Si el tipo de cultivo no existe, lanza excepción.
4	Si no hay stock suficiente de semillas, lanza excepción.
5	Si la cantidad sembrada es inválida, lanza excepción.

3.2.2. CU02 - Programación de actividades agrícolas

Especificación del caso de uso

Código	CU02
Nombre	Programación de actividades agrícolas
Descripción	Permite programar actividades de mantenimiento para cultivos específicos.
Actores	ACT01
Requerimiento asociado	RF2: Programar actividades agrícolas
Precondición	Debe existir al menos una siembra activa.

Secuencia normal

Paso	Acción
1	El empleado selecciona un cultivo activo.
2	El empleado elige tipo de actividad y fecha.
3	El sistema valida que no haya conflicto de horarios.
4	El sistema registra la actividad programada.
5	El sistema confirma la programación.

3.2.3. CU03 - Registro de cosechas

Especificación del caso de uso

Código	CU03
Nombre	Registro de cosechas
Descripción	Permite registrar cosechas obtenidas de cultivos.
Actores	ACT01
Requerimiento asociado	RF4: Registrar cosechas
Precondición	Debe existir siembra madura para cosechar.

3.2.4. CU04 - Registro de ventas

Especificación del caso de uso

Código	CU04
Nombre	Registro de ventas
Descripción	Permite registrar ventas de productos cosechados.

Actores	ACT01
Requerimiento asociado	RF5: Gestionar ventas
Precondición	Debe existir stock de cosecha.

3.2.5. CU05 - Generación de alertas

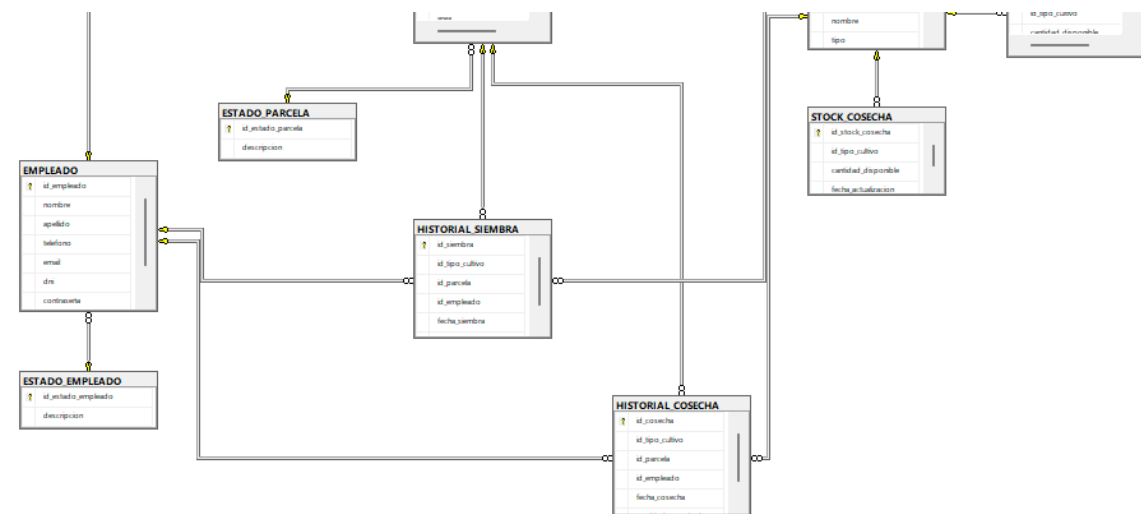
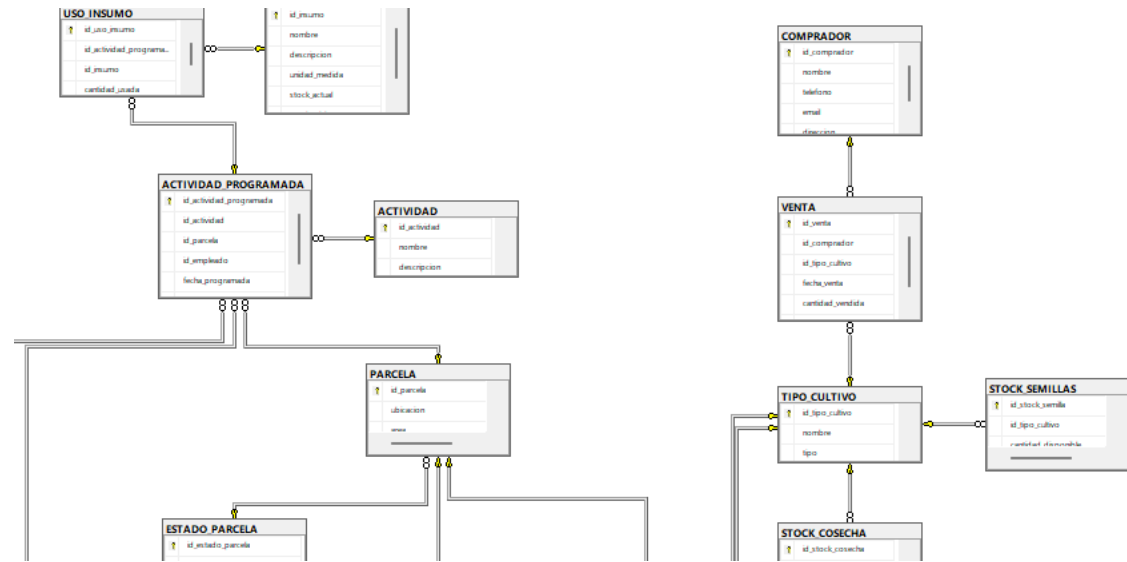
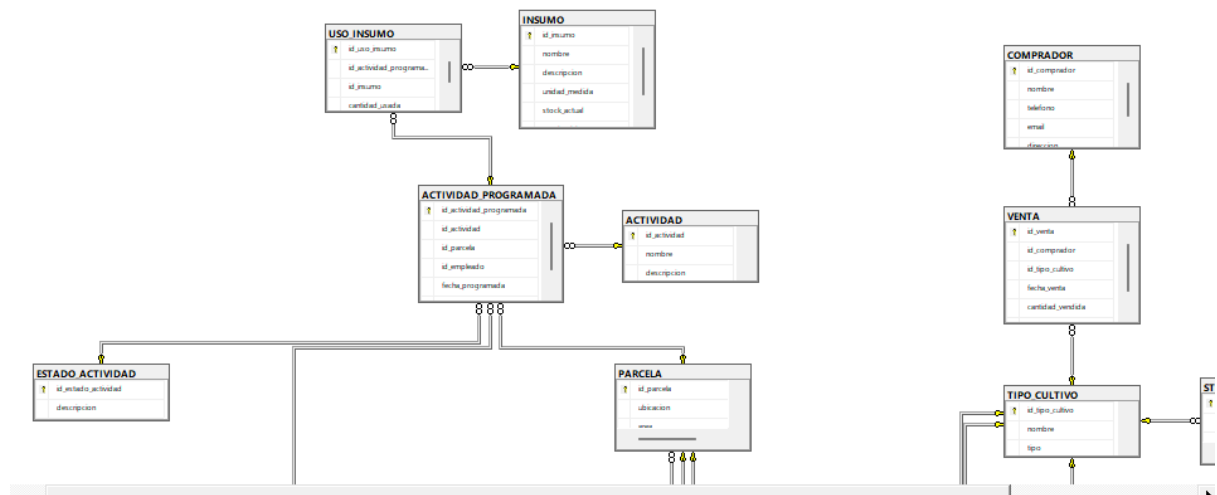
Especificación del caso de uso

Código	CU05
Nombre	Generación de alertas
Descripción	Sistema genera alertas automáticas para fechas críticas.
Actores	Sistema
Requerimiento asociado	RF3: Generar alertas
Precondición	El sistema está en funcionamiento.

Modelo Conceptual

El modelo conceptual del sistema AgroSan fue diseñado utilizando un diagrama Entidad-Relación, representando las principales entidades, atributos y relaciones del sistema agrícola:

- **Entidades principales:** Empleado, Parcela, Tipo_Cultivo, Historial_Siembra, Historial_Cosecha, Actividad_Programada, Stock_Semillas, Stock_Cosecha, Comprador, Venta, Alerta.
- **Relaciones clave:**
 - Un Empleado puede registrar múltiples siembras y cosechas
 - Una Parcela puede tener múltiples siembras a lo largo del tiempo
 - Cada siembra produce una cosecha
 - Las actividades programadas están asociadas a parcelas y empleados
 - Las ventas están vinculadas a compradores y tipos de cultivo
 - Los stocks de semillas y cosecha están asociados a tipos de cultivo



Modelo Relacional

- **Tablas principales:** Empleado, Parcela, Tipo_Cultivo, Historial_Siembra, Historial_Cosecha, Actividad_Programada, Stock_Semillas, Stock_Cosecha, Comprador, Venta.
- **Claves primarias y foráneas:** Cada tabla tiene su identificador único como clave primaria, y las relaciones se implementan mediante claves foráneas que mantienen la integridad referencial.

Diccionario de Datos

Esquema de Base de Datos - Gestión Agrícola (FarmaDB)

Este documento detalla la estructura de la base de datos, incluyendo campos, tipos, restricciones y relaciones, agrupadas por su función dentro del sistema de gestión agrícola.

1. Tablas de Usuarios y Clientes

Campo Clave	Tipo	Longitud	Restricciones	Relación	Descripción
EMPLEADO (Tabla 1)					Personal que opera el sistema y realiza tareas.
id_empleado	INT		PRIMARY KEY, IDENTITY		Identificador único del empleado.
nombre	NVARCHAR	50	NOT NULL		Nombre del empleado.
apellido	NVARCHAR	50	NOT NULL		Apellido del empleado.
telefono	NVARCHAR	20			Número de teléfono.

email	NVARCH AR	100			Correo electrónico.
dni	NVARCH AR	15	NOT NULL		Documento Nacional de Identidad.
contraseña	NVARCH AR	100	NOT NULL		Contraseña de acceso.
id_estado_empl eado	INT		NOT NULL	FK -> ESTADO_E MPLEADO	Estado actual del empleado (activo, inactivo, etc.).
COMPRADOR (Tabla 9)					Clientes a los que se les vende la cosecha.
id_comprador	INT		PRIMARY KEY, IDENTITY		Identificador único del comprador.
nombre	NVARCH AR	100	NOT NULL		Nombre o razón social del comprador.
telefono	NVARCH AR	20			Teléfono de contacto.
email	NVARCH AR	100			Correo electrónico.

direccion	NVARCH AR	255			Dirección del comprador.
-----------	--------------	-----	--	--	--------------------------

2. Tablas de Inventario y Ubicación

Campo Clave	Tipo	Longitud	Restricciones	Relación	Descripción
PARCELA (Tabla 2)					Unidades de terreno donde se realizan las siembras.
id_parcela	INT		PRIMARY KEY, IDENTITY		Identificador único de la parcela.
ubicacion	NVARCH AR	50	NOT NULL		Ubicación o nombre descriptivo de la parcela.
area	DECIMAL	(10,2)	NOT NULL		Área de la parcela en metros cuadrados (m²).
id_estado_parcela	INT		NOT NULL	FK -> ESTADO_PARCELA	Estado de la parcela (ocupada, libre, en mantenimiento).
TIPO_CULTIVO (Tabla 3)					Definición de los productos agrícolas.
id_tipo_cultivo	INT		PRIMARY KEY,		Identificador único del tipo de cultivo.

			IDENTITY		
nombre	NVARCHAR	50	NOT NULL		Nombre del cultivo (Ej: Tomate, Lechuga, Maíz).
tipo	NVARCHAR	50			Categoría (Ej: Hortaliza, Fruta, Cereal).
STOCK_SEMILLAS (Tabla 7)					Registro de semillas disponibles en el inventario.
id_stock_semilla	INT		PRIMARY KEY, IDENTITY		Identificador único de registro de stock de semillas.
id_tipo_cultivo	INT		NOT NULL	FK -> TIPO_CULTIVO	Tipo de cultivo al que pertenecen las semillas.
cantidad_disponible	DECIMAL	(10,2)	NOT NULL		Cantidad disponible de semillas en kg.
fecha_actualizacion	DATE		NOT NULL		Última fecha en que se actualizó el stock.
STOCK_COSECHA (Tabla 8)					Registro de producto cosechado disponible para venta.

id_stock_cosecha	INT		PRIMARY KEY, IDENTITY		Identificador único de registro de stock de cosecha.
id_tipo_cultivo	INT		NOT NULL	FK -> TIPO_CULTIVO	Tipo de cultivo cosechado.
cantidad_disponible	DECIMAL	(10,2)	NOT NULL		Cantidad disponible de cosecha en kg.
fecha_actualizacion	DATE		NOT NULL		Última fecha en que se actualizó el stock.

3. Tablas de Operaciones y Historial

Campo Clave	Tipo	Longitud	Restricciones	Relación	Descripción
HISTORIAL_SIEMBRA (Tabla 4)					Detalle de cada evento de siembra.
id_siembra	INT		PRIMARY KEY, IDENTITY		Identificador único de la siembra.
id_tipo_cultivo	INT		NOT NULL	FK -> TIPO_CULTIVO	Tipo de cultivo que se sembró.
id_parcela	INT		NOT NULL	FK -> PARCELA	Parcela donde se realizó la siembra.

id_empleado	INT		NOT NULL	FK -> EMPLEADO	Empleado responsable de la siembra.
fecha_siembra	DATE		NOT NULL		Fecha en que se realizó la siembra.
cantidad_sembrada	DECIMAL	(10,2)	NOT NULL		Cantidad de semilla sembrada en kg.
HISTORIAL_COSECHA (Tabla 5)					Detalle de cada evento de cosecha.
id_cosecha	INT		PRIMARY KEY, IDENTITY		Identificador único de la cosecha.
id_tipo_cultivo	INT		NOT NULL	FK -> TIPO_CULTIVO	Tipo de cultivo que se cosechó.
id_parcela	INT		NOT NULL	FK -> PARCELA	Parcela cosechada.
id_empleado	INT		NOT NULL	FK -> EMPLEADO	Empleado responsable de la cosecha.
fecha_cosecha	DATE		NOT NULL		Fecha de la cosecha.

cantidad_cosechada	DECIMAL	(10,2)	NOT NULL		Cantidad total cosechada en kg.
ACTIVIDAD_PROGRAMADA (Tabla 6)					Registro de tareas pendientes o realizadas.
id_actividad_programada	INT		PRIMARY KEY, IDENTITY		Identificador único de actividad.
id_actividad	INT		NOT NULL	FK -> ACTIVIDAD	Tipo de actividad (Ej: Riego, Fertilización).
id_parcela	INT		NOT NULL	FK -> PARCELA	Parcela donde se llevará a cabo la actividad.
id_empleado	INT		NOT NULL	FK -> EMPLEADO	Empleado asignado para realizar la actividad.
fecha_programada	DATE		NOT NULL		Fecha límite o programada para la actividad.
id_estado_actividad	INT		NOT NULL	FK -> ESTADO_ACTIVIDAD	Estado de la actividad (pendiente, en progreso, completada).

VENTA (Tabla 10)					Detalle de las transacciones de producto cosechado.
id_venta	INT		PRIMARY KEY, IDENTITY		Identificador único de venta.
id_comprador	INT		NOT NULL	FK -> COMPRADOR	Comprador que realizó la adquisición.
id_tipo_cultivo	INT		NOT NULL	FK -> TIPO_CULTIVO	Producto (cultivo) vendido.
fecha_venta	DATE		NOT NULL		Fecha en que se cerró la venta.
cantidad_vendida	DECIMAL	(10,2)	NOT NULL		Cantidad vendida en kg.
precio_unitario	DECIMAL	(10,2)	NOT NULL		Precio acordado por kg.

IMPLEMENTACIÓN DE LA BASE DE DATOS

Creación de la Base de Datos

Para crear la base de datos AgroSanDB se debe utilizar la instrucción CREATE DATABASE, como se ilustra a continuación:

```
USE master;
GO

-- Si la base de datos ya existe, eliminarla
IF EXISTS (SELECT name FROM sys.databases WHERE name = N'AgroSanDB')
BEGIN
    ALTER DATABASE AgroSanDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE AgroSanDB;
END
GO

-- Crear la nueva base de datos
CREATE DATABASE AgroSanDB;
GO

USE AgroSanDB;
GO
```

La estructura completa de tablas se encuentra definida en el archivo AgroSan.sql del proyecto, incluyendo todas las restricciones de integridad referencial y tipos de datos apropiados para cada campo.

CÓDIGO FUENTE

Se presentan las referencias a los archivos de código fuente más importantes del sistema AgroSan. En lugar de copiar el código completo, se indican las rutas y funciones principales de cada módulo:

1) Servicio de Siembra (RF1)

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/SiembraService.java
- **Funciones principales:**
 - registrarSiembra(): Registra nueva siembra con validaciones y cálculo automático de fecha de cosecha
 - listarSiembras(): Lista todas las siembras con información detallada
 - listarSiembrasPorParcela(): Filtra siembras por parcela específica
 - validarTipoCultivoExiste(): Valida existencia de tipo de cultivo

- validarCantidadSembrada(): Valida cantidad positiva
- obtenerDiasCosechaPorTipo(): Calcula días estimados según tipo de cultivo
- actualizarStockSemillas(): Reduce stock al sembrar
- hayStockSuficiente(): Verifica disponibilidad de semillas

2) Controlador REST de Siembra

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/rest/SiembraRest.java
- **Endpoints principales:**
 - POST /agrosan/siembra/regar: Registra nueva siembra
 - GET /agrosan/siembra/listar: Lista todas las siembras
 - GET /agrosan/siembra/parcela/{idParcela}: Lista siembras por parcela

3) Servicio de Cosecha (RF4)

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/CosechaService.java
- **Funciones principales:**
 - registrarCosecha(): Registra cosecha con validación de siembra activa
 - listarCosechas(): Lista todas las cosechas
 - listarCosechasPorParcela(): Filtra cosechas por parcela
 - actualizarStockCosecha(): Aumenta stock al cosechar
 - validarSiembraExiste(): Valida que haya siembra para cosechar

4) Controlador REST de Cosecha

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/rest/CosechaRest.java
- **Endpoints principales:**
 - POST /agrosan/cosecha/regar: Registra nueva cosecha
 - GET /agrosan/cosecha/listar: Lista todas las cosechas
 - GET /agrosan/cosecha/parcela/{idParcela}: Lista cosechas por parcela

5) Servicio de Actividades (RF2)

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/ActividadService.java
- **Funciones principales:**
 - programarActividad(): Programa nueva actividad agrícola
 - listarActividadesPendientes(): Lista actividades por completar
 - listarActividadesHoy(): Lista actividades del día actual
 - marcarCompletada(): Marca actividad como finalizada
 - validarCruceHorarios(): Evita conflictos de programación

6) Controlador REST de Actividades

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/rest/ActividadRest.java
- **Endpoints principales:**
 - POST /agrosan/actividad/programar: Programa nueva actividad
 - GET /agrosan/actividad/pendientes: Lista actividades pendientes
 - GET /agrosan/actividad/hoy: Lista actividades de hoy

7) Servicio de Ventas (RF5)

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/VentaService.java

- **Funciones principales:**

- registrarVenta(): Registra venta con validación de stock
- listarVentas(): Lista todas las ventas
- listarVentasPorComprador(): Filtra ventas por comprador
- validarStockSuficiente(): Verifica stock disponible
- actualizarStockVenta(): Reduce stock al vender
- calcularTotal(): Calcula total de venta

8) Controlador REST de Ventas

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/rest/VentaRest.java
- **Endpoints principales:**
 - POST /agrosan/venta/registrar: Registra nueva venta
 - GET /agrosan/venta/listar: Lista todas las ventas
 - GET /agrosan/venta/comprador/{idComprador}: Lista ventas por comprador

9) Servicio de Alertas (RF3)

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/AlertaService.java
- **Funciones principales:**
 - generarAlertasCosecha(): Genera alertas 7 días antes de cosecha
 - generarAlertasActividades(): Genera alertas de actividades pendientes
 - generarAlertasStock(): Genera alertas de stock bajo
 - listarAlertasActivas(): Lista alertas actuales
 - marcarLeida(): Marca alerta como leída

10) Controlador REST de Alertas

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/rest/AlertaRest.java
- **Endpoints principales:**
 - GET /agrosan/alerta/cosechas: Lista alertas de cosecha próxima
 - GET /agrosan/alerta/actividades: Lista alertas de actividades
 - GET /agrosan/alerta/stock: Lista alertas de stock bajo

11) Servicio de Empleados

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/EmpleadoService.java
- **Funciones principales:**
 - validarEmpleadoActivo(): Valida empleado existente y activo
 - autenticarEmpleado(): Valida credenciales de login
 - listarEmpleados(): Lista empleados del sistema

12) Servicio de Parcelas

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/service/ParcelaService.java
- **Funciones principales:**
 - validarParcelaActiva(): Valida parcela existente y activa
 - listarParcelas(): Lista parcelas disponibles
 - listarParcelasActivas(): Lista solo parcelas activas

DTOs (Data Transfer Objects)

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/dto/
 - SiembraDto.java
 - CosechaDto.java
 - ActividadProgramadaDto.java
 - VentaDto.java
 - EmpleadoDto.java
 - ParcelaDto.java
 - CompradorDto.java
 - TipoCultivoDto.java
 - ErrorResponse.java

Configuración de Base de Datos

- **Ubicación:** AgroSan/src/main/resources/application.properties
- Contiene configuración de conexión a SQL Server con JDBC.

Clase Principal

- **Ubicación:** AgroSan/src/main/java/pe/edu/uni/app/AgroSanApplication.java
- Clase principal con anotación @SpringBootApplication que inicia la aplicación.

CONCLUSIONES

- El diseño y estructura de una base de datos relacional sólida resultó fundamental para el desarrollo del sistema AgroSan, ya que permitió una adecuada gestión del ciclo agrícola completo, manteniendo relaciones consistentes entre entidades como empleados, parcelas, siembras, cosechas y ventas. Esto facilitó transacciones precisas y consultas eficientes para la toma de decisiones operativas.
- Las pruebas realizadas en los distintos módulos (siembra, cosecha, actividades, ventas, alertas) fueron clave para identificar errores lógicos y de integración. Estas pruebas no solo garantizaron que el sistema funcione como se esperaba, sino que también aseguraron la robustez de las operaciones ante entradas inválidas o inconsistencias en los datos agrícolas.
- Las validaciones implementadas en el sistema permiten preservar la integridad de los datos agrícolas verificando la existencia de registros asociados, como parcelas activas, empleados disponibles y stock suficiente. Además, se aplican controles sobre formatos, rangos y duplicados, reforzando la fiabilidad del sistema para entornos de producción agrícola.
- La automatización de procesos como el cálculo automático de fechas estimadas de cosecha, actualización de inventarios y generación de alertas demuestra que el sistema puede adaptarse a situaciones reales de gestión agrícola. Esto mejora la experiencia del usuario y reduce errores humanos en cálculos críticos para la producción.
- El trabajo en equipo y la correcta planificación del desarrollo permitieron la creación de un sistema escalable y modular. Su arquitectura basada en Spring Boot y el patrón de diseño en capas (Controller-Service-Repository) facilita la incorporación futura de funcionalidades como integración con sensores IoT, análisis predictivo de cosechas o módulos de gestión financiera, asegurando la sostenibilidad del sistema a largo plazo.

LECCIONES APRENDIDAS

- Aprendimos a validar correctamente los datos agrícolas críticos como cantidades sembradas/cosechadas, existencia de stock de semillas, disponibilidad de parcelas y empleados activos. Estas validaciones fueron clave para evitar inconsistencias en el ciclo productivo y mejorar la trazabilidad de las operaciones.
- Supimos aprovechar las capacidades de SQL Server para implementar restricciones de integridad referencial que garantizan la consistencia de datos entre siembras, cosechas y ventas. El uso de transacciones ACID resultó fundamental para mantener la integridad en operaciones complejas que involucran múltiples tablas.
- Comprendimos la importancia de una arquitectura modular en aplicaciones Spring Boot, separando claramente las responsabilidades entre controladores REST (endpoints HTTP), servicios de negocio (lógica de validación y procesamiento) y repositorios (acceso a datos). Esta separación facilita el mantenimiento y testing del código.
- Aprendimos a manejar excepciones de manera efectiva en aplicaciones Java, creando respuestas de error estandarizadas que proporcionan información clara sobre qué salió mal y dónde, mejorando significativamente la experiencia de desarrollo y debugging.
- Entendimos la importancia del manejo de fechas en sistemas agrícolas, implementando cálculos automáticos para fechas estimadas de cosecha basados en tipos de cultivo, y validaciones que evitan conflictos de horarios en actividades programadas.
- Experimentamos con el desarrollo de APIs RESTful bien documentadas, utilizando Postman para testing exhaustivo de todos los endpoints, lo que nos permitió validar no solo los casos de éxito sino también múltiples escenarios de error.

BIBLIOGRAFÍA

1. Spring Framework Documentation. (2024). Spring Boot Reference Documentation.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
2. Microsoft SQL Server Documentation. (2024). SQL Server Technical Documentation.
<https://docs.microsoft.com/en-us/sql/sql-server/>
3. Postman API Platform. (2024). Postman Learning Center.
<https://learning.postman.com/>
4. Oracle Java Documentation. (2024). Java SE Documentation.
<https://docs.oracle.com/en/java/javase/>
5. Bauer, C., & King, G. (2023). Java Persistence with Hibernate. Manning Publications.
6. Evans, E. (2023). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.
7. Fowler, M. (2023). Patterns of Enterprise Application Architecture. Addison-Wesley.
8. Hunt, A., & Thomas, D. (2023). The Pragmatic Programmer: Your Journey to Mastery. Addison-Wesley.

ANEXOS

Anexo 1: Diagrama de Arquitectura del Sistema

CAPA DE PRESENTACIÓN

REST CONTROLLERS

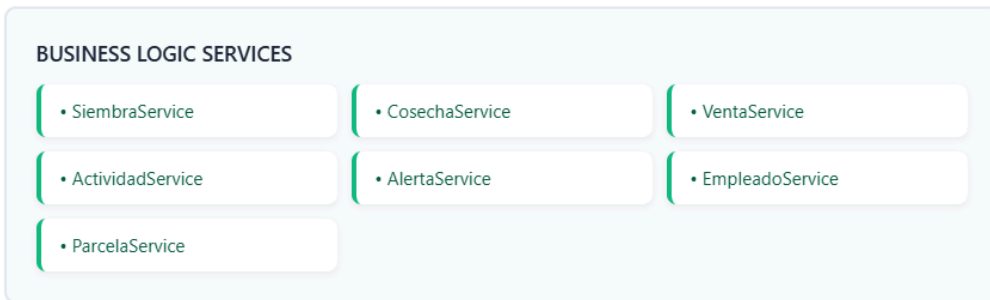
- | | | |
|-----------------|---------------|----------------|
| • SiembraRest | • CosechaRest | • VentaRest |
| • ActividadRest | • AlertaRest | • EmpleadoRest |
| • ParcelaRest | | |

CAPA DE SERVICIO

BUSINESS LOGIC SERVICES

- | | | |
|--------------------|------------------|-------------------|
| • SiembraService | • CosechaService | • VentaService |
| • ActividadService | • AlertaService | • EmpleadoService |
| • ParcelaService | | |

CAPA DE SERVICIO



CAPA DE DATOS



Anexo 2: Colección de Postman

La colección completa de Postman se encuentra en el archivo `AgroSan_Postman_Collection.json` e incluye:

- 7 folders organizados por módulo (Empleados, Parcelas, Siembra, Cosecha, Ventas, Actividades, Alertas)
- Más de 50 requests de prueba
- Casos de éxito y casos de error para cada funcionalidad
- Variables de entorno configuradas
- Headers y bodies JSON preparados

Para importar la colección:

1. Abrir Postman
2. Click en "Import"
3. Seleccionar archivo `AgroSan_Postman_Collection.json`
4. Verificar que las variables de entorno estén configuradas
5. Ejecutar Spring Boot
6. Probar los endpoints en orden sugerido

Anexo 3: Scripts de Base de Datos

- AgroSan.sql: Script completo de creación de base de datos
- AgroSan_Data.sql: Script de inserción de datos de prueba

Fin del Informe - Sistema AgroSan